



File Format Specification

Version 0.9 · Draft 5

HUPO-PSI Recommendation — Working Draft

HUPO Proteomics Standards Initiative

Generated 2026-06-12 · spec 0631167

Contents

1	File Format Specification	4
1.1	Abstract	4
1.2	What you'll find here	4
1.3	Anatomy of an archive	5
1.4	Reference implementations	5
1.5	Status of this document	6
2	Overview	7
2.1	Components of an archive	7
3	Motivation	8
3.1	Description of the need	8
3.2	Issues to be addressed	8
4	Relationship to Other Specifications	9
4.1	Controlled vocabularies	9
5	Notational Conventions	10
5.1	Code samples	10
6	Anatomy of a Parquet File	11
6.1	The schema	11
6.2	The metadata key-value pairs	11
6.3	The columnar data	11
6.3.1	Row groups and pages	11
6.3.2	Index levels	12
7	Container & Archive	13
7.1	ZIP archives	13
7.1.1	Why not TAR?	13
7.2	Unpacked archives	13
7.3	Encryption	13
8	Packed Parallel Metadata Tables	14
8.1	Controlled Vocabulary Terms	14
8.1.1	The parameters list	15
8.1.2	Typing parameter values	15
8.1.3	Column name inflection	15
8.2	Null semantics for metadata	16
9	Signal Data Model	17
9.1	Arrays and columns	17
9.2	The Array Index	17
9.2.1	Buffer format	18
9.2.2	Buffer priority and naming	19
9.3	Data arrays, encoding, transformations, and Parquet	19
9.3.1	Zero run stripping	19
9.3.2	Null marking	20
9.3.3	Null semantics for signal data	23

9.4 Why a top-level node?	23
10 Point Layout	24
11 Chunked Layout	26
11.1 Column naming rules	26
11.2 Splitting data into chunks	26
11.3 Chunk encodings	28
11.3.1 Basic encoding	28
11.3.2 Delta encoding	29
11.3.3 Numpress linear encoding	30
11.4 Opaque array transforms	30
11.5 Reading a single entry from the chunked encoding	30
12 Auxiliary Data Arrays	31
13 The Index File – mzpeak_index.json	33
13.1 File-level metadata	34
14 Data Kinds	35
14.1 Adding a new data kind	35
15 Entity Types	36
15.1 Decided at HUPO-PSI 2026 (Rome)	36
15.2 Adding a new entity type	37
16 Spectra	38
16.1 Spectrum signal data – spectra_data.parquet	38
16.1.1 Recommended Parquet encodings	38
16.2 Spectrum peak data – spectra_peaks.parquet	39
16.3 Spectrum metadata – spectra_metadata.parquet	39
16.3.1 spectrum (group)	39
16.3.2 scan (group)	39
16.3.3 precursor (group)	40
16.3.4 selected_ion (group)	40
17 Chromatograms	41
17.1 Chromatogram signal data – chromatograms_data.parquet	41
17.1.1 Recommended Parquet encodings	41
17.2 Chromatogram metadata – chromatograms_metadata.parquet	41
17.2.1 chromatogram (group)	41
17.2.2 precursor (group)	42
17.2.3 selected_ion (group)	42
18 Wavelength Spectra	43
18.1 Wavelength spectrum signal data – wavelength_spectra_data.parquet	43
18.2 Wavelength spectrum metadata – wavelength_spectra_metadata.parquet	43
18.2.1 spectrum (group)	43
18.2.2 scan (group)	44
19 Implementations	45
19.1 Rust – reference implementation	45
19.2 Python	45
19.3 R	45
19.4 C	45
19.5 JavaScript / TypeScript	46
20 Tools	47
20.1 Converters	47
20.2 Validation	47
20.3 Viewers	47
20.4 Example data	47

21 Glossary	48
22 References	50
23 Authors & Contributors	51
23.1 Authors	51
23.2 Contributors	51
23.3 How to contribute	51
24 Intellectual Property	52
24.1 Intellectual property statement	52
24.2 Trademark	52
24.3 Copyright notice	52

Chapter 1

File Format Specification

An open, columnar, random-access file format for storing mass spectra, chromatograms, and rich instrument metadata — designed to succeed mzML for modern, large-scale mass spectrometry.

Status: Draft Version 0.9 · Draft 5 Built on Apache Parquet CC-BY-ND 4.0

WARNING — This is a working draft

This document is a **DRAFT** under active revision and has **not** yet been ratified through the [HUPO-PSI Document Process](#). Content, schemas, and controlled-vocabulary terms may change. Feedback is explicitly solicited — please [open an issue](#).

1.1 Abstract

mzPeak is an open, next-generation file format for mass spectrometry data. Advances in MS instrumentation — higher resolution, faster scan rates, greater sensitivity, and the growing use of ion mobility and imaging — have sharply increased the volume and dimensionality of the data produced in proteomics, metabolomics, and lipidomics. Established open formats such as mzML and imzML increasingly struggle with the resulting file sizes, data-access patterns, and metadata demands, while vendor-specific formats offer faster access at the cost of interoperability and long-term archival stability. mzPeak is designed to address these needs as a community standard; the motivation and design rationale are set out in the accompanying white paper (1).

The format takes a hybrid approach: an archive of [Apache Parquet](#) tables — packaged in an uncompressed ZIP container, or an unpacked directory, together with a small JSON index — that stores numerical signal data as compact, columnar binary while keeping metadata both human- and machine-readable and anchored in the PSI-MS controlled vocabulary. Building on the widely available Apache Parquet and Apache Arrow libraries rather than a single core implementation, the on-disk structure is language-independent. Uncompressed archive members and a required Parquet page index let a reader locate and decode a single spectrum without parsing an entire run, the basis for random access from local disk, object storage, or over HTTP. This document specifies that format: its container, signal-data layouts, index file, and metadata model.

1.2 What you'll find here

- **Introduction**

Why mzPeak exists, what problems it solves, and a high-level tour of an archive.

 Overview & motivation

- **Foundations**

A short Parquet primer and how multiple Parquet files are bundled into one archive.

 Parquet & containers

- **Data Layouts**

Packed metadata tables, the array index, and the point & chunked signal layouts.

:octicons-arrow-right-24: Layouts

- **Archive Organization**

The `mzpeak_index.json` file, data kinds, and entity types that organise a run.

:octicons-arrow-right-24: The index file

- **File Schemas**

Concrete column-level schemas for spectra, chromatograms, and wavelength spectra.

:octicons-arrow-right-24: Schemas

- **Implementations**

The read/write libraries — Rust, Python, R, C#, and JavaScript/TypeScript.

:octicons-arrow-right-24: Implementations

- **Tools**

Converters, the conformance validator, browser viewers, and example data.

:octicons-arrow-right-24: Tools

- **Reference**

Glossary, bibliography, authorship, and the intellectual-property statement.

:octicons-arrow-right-24: Reference

1.3 Anatomy of an archive

An mzPeak archive (`.mzpeak`) is a small set of named files:

File	Contents
<code>mzpeak_index.json</code>	Manifest describing every file in the archive by entity type and data kind.
<code>spectra_metadata.parquet</code>	Spectrum-level and file-level metadata (descriptions, scans, precursors, selected ions).
<code>spectra_data.parquet</code>	Spectrum profile signal data, in point or chunked layout.
<code>spectra_peaks.parquet</code> (<i>optional</i>)	Spectrum centroid signal, stored separately from profile data.
<code>chromatograms_metadata.parquet</code>	Chromatogram-level and file-level metadata.
<code>chromatograms_data.parquet</code>	Chromatogram signal data, in point or chunked layout.

Additional Parquet files may be added to cover further modalities (for example wavelength spectra) as the format grows.

1.4 Reference implementations

mzPeak is backed by five independent, from-scratch implementations (not bindings to a single core):

- **Rust** — read/write reference implementation · [HUPO-PSI/mzPeak](#)
- **Python** — read-only, zero-copy Arrow/Pandas API
- **R** — read-only, `dplyr`-compatible access
- **C#** — read/write · [HUPO-PSI/mzPeak.NET](#)
- **JavaScript / TypeScript** — read-only, runs in the browser, Node, and Deno · [online viewer](#)

See Implementations for details, and Tools for converters, the conformance validator, and viewers.

1.5 Status of this document

This document provides information to the proteomics community about the mzPeak file format. Distribution is unlimited. It will be ratified via the HUPO Proteomics Standards Initiative (PSI) Document Process, and any alterations **MUST** also follow the HUPO-PSI Document Process.

Version: Draft 5 of version 0.9

Chapter 2

Overview

mzPeak is an open, next-generation file format for mass spectrometry data. Advances in MS instrumentation — higher resolution, faster scan rates, greater sensitivity, and the growing use of ion mobility and imaging — have sharply increased the volume and dimensionality of the data produced in proteomics, metabolomics, and lipidomics. Established open formats such as mzML and imzML increasingly struggle with the resulting file sizes, data-access patterns, and metadata demands, while vendor-specific formats offer faster access at the cost of interoperability and long-term archival stability. mzPeak is designed to address these needs as a community standard; the motivation and design rationale are set out in the accompanying white paper (1) and summarised under Motivation.

The format takes a hybrid approach: an archive of [Apache Parquet](#) tables — packaged in an uncompressed ZIP container, or an unpacked directory, together with a small JSON index — that stores numerical signal data as compact, columnar binary while keeping metadata both human- and machine-readable and anchored in the PSI-MS controlled vocabulary. Building on the widely available Apache Parquet and Apache Arrow libraries rather than a single core implementation, the on-disk structure is language-independent. Uncompressed archive members and a required Parquet page index let a reader locate and decode a single spectrum without parsing an entire run, the basis for random access from local disk, object storage, or over HTTP. This document specifies that format: its container, signal-data layouts, index file, and metadata model.

While mzPeak draws on prior art like [mzML](#), it is not a direct re-implementation of mzML in a Parquet table. It re-uses concepts such as controlled vocabularies where feasible, alongside arbitrary additional user metadata. For the conventions used in the normative text, see Notational Conventions.

2.1 Components of an archive

- **mzpeak_index.json** — definition of the files present in the archive, encoded as JSON. Resolving files by controlled terms is easier than matching file names. See The Index File.
- **spectra_metadata.parquet** — spectrum-level and file-level metadata, including spectrum descriptions, scans, precursors, and selected ions, using packed parallel tables.
- **spectra_data.parquet** — spectrum **profile** signal data, in point layout or chunked layout, which have different size and random-access characteristics.
- **spectra_peaks.parquet** (*optional*) — spectrum **centroids**, stored explicitly separately from the profile signal in `spectra_data.parquet`. This is how mzPeak holds both profile and centroid versions of the same spectra. This file may not always be present.
- **chromatograms_metadata.parquet** — chromatogram-level and file-level metadata, including chromatogram descriptions, precursors, and selected ions.
- **chromatograms_data.parquet** — chromatogram signal data, in point or chunked layout. Intensity measures with different units may be stored in parallel.

Other Parquet files may be added to cover additional modalities as needed, such as wavelength spectra.

Chapter 3

Motivation

3.1 Description of the need

mzML has served the mass spectrometry community well as an open, vendor-neutral interchange format. As instruments have become faster and experiments larger — ion-mobility separations, imaging, data-independent acquisition — several of mzML’s design choices have become limiting:

- **Size and access are at odds.** mzML stores binary arrays as base64-encoded text inside XML. The encoding inflates files, and the usual remedy (whole-file gzip) destroys the random access needed to read a single spectrum or an extracted-ion chromatogram without decompressing everything.
- **Imaging data fits poorly.** Spatial coordinates have to be bolted on; imzML exists but integrates awkwardly with the rest of the ecosystem.
- **Vendor metadata is lost.** Much of what an instrument records has no place in the mzML model and is discarded at conversion time.
- **Profile and centroid cannot coexist.** A single mzML spectrum cannot carry both its raw profile signal and a centroided peak list at the same time.
- **No encryption.** mzML offers no mechanism for protecting parts of a file.

mzPeak is designed to remove these limitations while remaining open, language-agnostic, and grounded in the controlled vocabularies the community already uses.

3.2 Issues to be addressed

The format is shaped by a small number of concrete goals:

1. **Compact storage with preserved random access** — files smaller than mzML (often by half or more) that can still be sliced by spectrum, m/z range, or time range without decoding the whole file.
2. **Fast analytical queries** — extracted-ion chromatograms and similar reads served efficiently through columnar page indices.
3. **Profile and centroid signal** for the same spectrum, stored side by side.
4. **Richer metadata** — room for the vendor and acquisition metadata mzML cannot express, without forcing it into a rigid schema.
5. **Additional modalities** — ion mobility, imaging coordinates, wavelength spectra, and diagnostic traces, with room to grow.
6. **Optional encryption** of sensitive parts of an archive.
7. **A stable, cross-language data substrate** that many implementations can read and write independently.

Chapter 4

Relationship to Other Specifications

This format is not developed in isolation. It is designed to be complementary to — and used in conjunction with — several existing and emerging community models:

- **PSI Universal Spectrum Identifier (USI)** — a universal mechanism for referring to a specific spectrum in public repositories. mzPeak uses USIs in the `scan.spectrum_reference` field for spectra drawn from external sources (see Spectrum metadata).
- **mzML** — the current PSI recommendation for storing raw or processed mass spectrometry data. mzPeak draws on the mzML data model and re-uses concepts such as controlled vocabularies where feasible.
- **SDRF (Sample and Data Relationship Format)** — a representation of sample metadata. SDRF known at acquisition time may be carried alongside the data in an mzPeak archive.
- **imzML** — the mzML-derived standard for mass-spectrometry imaging. mzPeak addresses imaging within the same archive via spatial coordinates in the spectrum metadata (see Entity Types).

4.1 Controlled vocabularies

mzPeak metadata is anchored in the **PSI-MS controlled vocabulary**. The controlled vocabularies an archive relies on — with their versions and source URIs — are declared in the archive's `cv_list` (see file-level metadata), so a reader can resolve every CURIE unambiguously and reproducibly.

Chapter 5

Notational Conventions

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in [RFC 2119](#).

Throughout the specification, controlled-vocabulary terms are written as CURIes of the form `MS:1000511` and, where helpful, linked to their definition in the [PSI-MS controlled vocabulary](#). When a term name is *inflected* into a Parquet column name, it appears in code font, e.g. `MS_1000511_ms_level` — see Column Name Inflection.

5.1 Code samples

This document describes both a file format and a set of suggested algorithms for preparing data to be stored in that format. The original author (Joshua Klein) includes snippets of Python code to illustrate these operations, under the assumption that most technical programmers will be able to read Python and that Python is effectively executable pseudocode.

The snippets rely on three components:

- **abstract base classes** from the Python standard library, used for type annotations;
- **NumPy** v2.1, for array operations that are assumed to be understandable; and
- **PyArrow** v20.0, for operations on Apache Arrow arrays, which are conceptually equivalent to the in-memory representation of data stored in Parquet files.

NOTE — Algorithms are illustrative, not normative

Except where the prose explicitly uses RFC 2119 keywords, the code samples describe *one* correct way to produce conformant data. Implementations are free to use any method that yields an equivalent, conformant result.

Chapter 6

Anatomy of a Parquet File

This is a minimal overview of Parquet. For full details, see the [Apache Parquet documentation](#).

6.1 The schema

Parquet files contain a physical data schema defining how their data columns are encoded as bytes on disk. This schema supports arbitrary levels of nullability, nesting (groups), and repetition (lists). These physical data types may also be mapped to one or more *logical types*.

There is a broader many-to-many mapping between Parquet schemas and [Apache Arrow](#) schemas. Arrow supports many types that Parquet does not, but the two share a common abstraction of columnar storage with per-value nullability. Although they represent these concepts differently, conversion between them is straightforward.

6.2 The metadata key–value pairs

[figure: Parquet file layout schematic — omitted]

The Parquet file layout: row groups of column chunks, followed by a footer containing the schema, offsets, indices, and user metadata.

At the end of a Parquet file is a footer containing user-defined metadata along with the file's schema, offsets, and search indices. This user-defined metadata is stored as key–value pairs, which makes it well suited to carrying light-weight, immediately useful metadata that does not make sense to force into the data columns. mzPeak uses this footer extensively — most notably for the array index.

6.3 The columnar data

Parquet is a strongly typed binary columnar format with layered, blocked compression that permits a degree of random access.

WARNING — Support both 32-bit and 64-bit collection types

Because some languages have no concept of 64-bit addresses, all implementations **MUST** handle both the `large_*` and `non-large_*` variants of collection types: `list`, `string`, and `binary`.

6.3.1 Row groups and pages

A Parquet file is divided into **row groups**, each of which stores a horizontal slice of every column as a **column chunk**. Column chunks are further divided into **pages**, the smallest unit of compression and encoding. This hierarchy is what lets a reader retrieve a narrow slice of a large file: it can locate the row group and pages that hold the rows of interest and read only those.

Choosing row-group and page sizes trades compression ratio against random-access granularity – larger groups compress better but make fine-grained reads coarser.

INFO – Open item – tuning guidance

Concrete recommendations for balancing compression against random-access granularity (row-group size, page size, dictionary page size) are still being collected and will be expanded in a future revision.

6.3.2 Index levels

When writing an mzPeak archive, the writer **MUST** write a [page index](#). Most libraries that write Parquet support writing the page index even if they do not themselves use it when reading. The page index records per-page value ranges, which is what makes predicate-driven reads (for example, “only pages whose spectrum_index covers 5000–5100”) efficient.

Chapter 7

Container & Archive

An mzPeak archive bundles several Parquet files and a JSON index under a single name. That bundle can be delivered three ways: as a ZIP archive, as an unpacked directory, or as a remote prefix (for example an HTTP server supporting range requests, S3, FTP, or WebDAV).

7.1 ZIP archives

To pack multiple Parquet tables together under a single file name on disk, we need a container format. mzPeak uses the [ZIP](#) archive to bundle the member files together. A ZIP file begins with a header containing its magic bytes, followed by a sequence of (header, file) blocks, and terminates with a **central directory** that records how to find each member.

Files in a ZIP may be stored compressed or uncompressed.

WARNING — Members MUST be stored uncompressed

When mzPeak is stored in a ZIP, it **MUST** store its member files *uncompressed*. Uncompressed members can be read directly without an intervening decompression step to reveal the Parquet file, and each Parquet file already contains layered compression superior to that of a typical ZIP compressor.

7.1.1 Why not TAR?

TAR archives are designed for linear traversal: to learn what files an archive contains you must hop from header entry to header entry until you reach the end. Compared with ZIP's central directory, this is less efficient and more expensive on object stores. TAR also has no per-file encryption, which would make protecting the parts of an archive that are *not* Parquet files harder.

7.2 Unpacked archives

If an mzPeak archive is stored as an unpacked directory, the directory name is treated as the name of the run.

7.3 Encryption

Because Parquet supports [modular encryption](#), individual Parquet files — and even individual columns or the footer metadata — can be encrypted while leaving the archive readable as a whole.

QUESTION — Open item — index visibility vs. encryption

Anything placed in `mzpeak_index.json` is necessarily visible in cleartext to all readers unless ZIP encryption is used, and ZIP encryption is widely known to be flawed and inconsistently implemented. Metadata inside a Parquet footer's key-value pairs *can* be encrypted. The index is JSON for convenience and for easy access from scripting languages; whether sensitive fields should move into encryptable Parquet metadata is an open design question.

Chapter 8

Packed Parallel Metadata Tables

The `spectra_metadata.parquet` and `chromatograms_metadata.parquet` files store multiple schemas *in parallel*. The root schema is made up of several branched “group” (Parquet) or “struct” (Arrow) facets, any of which may be null at any level. We borrow relational-database language — **primary key** and **foreign key** — to describe how the parallel tables interconnect.

The example below shows two rows of related MS1 and MS2 spectra. Treat `scan.source_index`, `precursor.source_index`, `precursor.precursor_index`, `selected_ion.source_index`, and `selected_ion.precursor_index` as foreign keys with respect to `spectrum.index`, the primary key:

- `precursor.source_index` refers to the spectrum that this precursor record belongs to.
- `precursor.precursor_index` refers to the spectrum that *is* the precursor of the spectrum referenced by `precursor.source_index`.
- Together, (`precursor.source_index`, `precursor.precursor_index`) forms a compound key.

Any of these columns may be null, meaning that such a record does not exist in the table. The same applies to the `selected_ion` facet.

spectrum			scan			precursor			selected_ion			
			MS_- 1000511_-	source_-	MS_- 1000616_- pre-set_ scan_- configuration	source_-	precursor	isolation_-	source_-	precursor_-	MS_- 1000744_- se- lected_ ion_mz	
index	id	time	ms_- level	index		index	index	window	index	index		
502	scan=502	20.51	1	502	3	...	503	502	{...}	503	502	233.5
503	scan=503	20.531	2	503	2	...	504	502	{...}	504	502	562.3
...												

8.1 Controlled Vocabulary Terms

Like mzML, mzPeak makes heavy use of controlled vocabularies to represent rich metadata. mzPeak uses CV terms in three ways:

1. **As columns.** When a term is used as a column name, that column’s values are either the defined value of the term’s expected type (e.g. via `has_value_type`) or a CURIE for a child of the column-name term. For example:
 - The column `MS_1000525_spectrum_representation` holds CURIEs for a child term — `MS:1000127` “centroid spectrum” or `MS:1000128` “profile spectrum” — as appropriate for the spectrum in that row.
 - The column `MS_1000511_ms_level` holds an integer value.
2. **As structural elements.** In several places — such as the array index — CURIEs reference named concepts that explain the semantics of a data structure without changing its shape.
3. **As pluggable metadata carriers** in parameters arrays, analogous to `<cvParam/>` in mzML. Every schema facet of a metadata table may carry a parameters column.

8.1.1 The parameters list

The parameters column may be present in any facet of a metadata table. It **MUST** be a list of the following schema:

```
optional group field_id=-1 parameters (List) {
  repeated group field_id=-1 list {
    optional group field_id=-1 item {
      optional group field_id=-1 value {
        optional int64   field_id=-1 integer;
        optional double  field_id=-1 float;
        optional binary  field_id=-1 string (String);
        optional boolean field_id=-1 boolean;
      }
      optional binary field_id=-1 accession (String);
      optional binary field_id=-1 name (String);
      optional binary field_id=-1 unit (String);
    }
  }
}
```

The parameters.list.item.value group has a slot for each data type, so a parameter can take exactly one of these value types; unused slots **MUST** be null. A parameter **MAY** carry a unit, given as a CV-term CURIE in parameters.list.item.unit. As with mzML's <userParam/>, an *uncontrolled* parameter may be stored simply by leaving parameters.list.item.accession empty.

NOTE – Naming and column promotion

- Parquet columns **MUST** be uniquely named, so if a parameter appears more than once in a single entry it **MUST** be stored in the parameters column.
- Writers are encouraged, when sufficient context is available, to promote parameters that are present in most rows of a table to *columns*. This is more space-efficient and enables predicate filtering.

QUESTION – Open item – lists and maps

Whether parameters values should also support list- or map-valued types is still under discussion.

8.1.2 Typing parameter values

When a term has a value and it is stored in the parameters list, the value **MUST** use one of the value types in the fixed schema above. When a term is instead promoted to its own column, the value may use any Parquet data type. This flexibility lets writers pick an appropriate size and precision, but it can create redundant reader code – a column expected to hold an integer might be written as an 8-, 16-, 32-, or 64-bit integer, signed or unsigned. Some languages handle this naturally with dynamic typing or generics; others need a separate implementation per physical type.

Recommended physical types:

- **Integral values that are *not* indices or identifiers** – prefer signed 32- or 64-bit, as needed to cover the value's domain.
- **Indices or identifiers represented as integers** – prefer unsigned 32- or 64-bit. Dictionary encoding reduces most cases to ~8 bits per variant on disk anyway.
- **Floating-point values** – prefer 64-bit doubles unless precision is truly of no concern. For repetitive values (e.g. collision energy), dictionary encoding drops the on-disk cost well below 32 bits per value.
- **Strings and lists** – prefer 64-bit offsets (“large strings”/“large lists”), but write code that supports both 32- and 64-bit offsets. This matters especially for strings, where the offset is a *byte* offset, not an item offset.

8.1.3 Column name inflection

When a CV-term concept is represented as a column, the column name **SHOULD** be constructed by the following inflection rules:

1. The base column name is `${CV_CODE}_${CV_ACCESSION}_${CLEANED_NAME}` where:
 1. `${CV_CODE}` identifies the controlled vocabulary itself — MS for PSI-MS, UO for the Units of Measurement Ontology.
 2. `${CV_ACCESSION}` is the accession number. For MS:1000016 “scan start time” this is 1000016.
 3. `${CLEANED_NAME}` is the term’s name with any character that is not valid in a Parquet column name replaced by `_`. The regular expression `/[^a-zA-Z0-9_\-\.\-]+/` matches all such characters in ASCII. For MS:1000016, the result is `MS_1000016_scan_start_time`.
 4. Because the string “m/z” appears so frequently, it **SHOULD** be rewritten as `mz` to avoid extra underscores. For MS:1000504 “base peak m/z”, the result is `MS_1000504_base_peak_mz`.
2. **If a single unit applies to every value in the column**, it **SHOULD** be appended as `_unit_${UNIT}-${CV_CODE}_${UNIT_${CV_ACCESSION}}`. For example, `MS_1000528_lowest_observed_mz_unit-MS_1000040` is MS:1000528 “lowest observed m/z” with unit MS:1000040 “m/z”.
3. **If the unit varies across the column**, it **SHOULD** be carried in a separate column whose name is the inflected name with `_unit` appended, whose values are the unit CURIEs.

8.2 Null semantics for metadata

A row value that is `null` is treated as absent — having no value. If a *foreign key* column is `null`, assume the corresponding record does not exist (as when an MS2 spectrum is stored without its MS1 precursor, as in MGF files, or in a slice of a run). If the *primary key* of a facet is `null`, the reader **SHOULD** skip that facet’s columns for that row.

A writer **SHOULD** minimise the number of interspersed all-null rows, though this is not strictly required. Minimising interspersed nulls improves compressibility. In the figures below, “Packed Tables” keeps the rows of each parallel facet contiguous, while “Sparse Tables” intermixes rows of nulls.



Chapter 9

Signal Data Model

9.1 Arrays and columns

In mass spectrometry it is common to speak of a spectrum *having* an m/z array, as a synonym for having been measured in the m/z dimension, with a parallel intensity array giving the abundance of the signal at each m/z. In mzML, the two dimensions may even use different physical data types on different spectra in the same file, and there are legitimate use-cases for that.

mzPeak can store array data in two ways:

- **As a column** in a signal data layout — the array is burned into the schema and registered in the array index. It is encoded directly by Parquet, subject to Parquet’s adaptive encoding and compression, and — crucially — can be searched and sliced via the page index.
- **As an auxiliary array** — stored in the associated metadata table’s `*.auxiliary_arrays` value for that entity’s row. Auxiliary arrays can be individually configured by the writer (custom compression, data type, decoding, or cvParams), but they **cannot** be searched or sliced without decoding the entire array, just as in mzML.

Currently, the first sorted array is assumed to be the axis around which all other values are arranged — *sorting rank 0*. Any arrays that are shorter or longer than this axis **SHOULD** be stored as auxiliary arrays instead.

WARNING — Re-sort unsorted ranked arrays

When writing, if an array that has a sorting rank is not sorted, the entry’s data arrays **MUST** be re-sorted accordingly. Failing to do so introduces integrity errors.

9.2 The Array Index

To annotate what kind of array a column *is*, mzPeak stores a JSON-serialised **array index** in the Parquet key-value metadata. It is a list of structures describing each array using controlled vocabulary. A column is part of the Parquet schema and must always exist with a homogeneous value type (or be null) for each row.

The array index is stored in the Parquet metadata of the data arrays or peaks files under `<entity_type>_array_index` — e.g. `spectrum_array_index` for spectra or `chromatogram_array_index` for chromatograms.

```
{
  "prefix": "point",
  "entries": [
    {
      "context": "spectrum",
      "path": "point.mz",
      "data_type": "MS:1000523",
      "array_type": "MS:1000514",
      "array_name": "m/z array",
      // This array describes a spectrum
      // Column path in the Parquet schema
      // CV term for the data type of this array
      // CV term for the array itself
      // Human-readable name; also where a custom
      // 'non-standard array' name is stored
    }
  ]
}
```

```

    "unit": "MS:1000040",           // Values are in m/z
    "buffer_format": "point",       // This column uses the point layout
    "transform": null,              // No transformation applied
    "data_processing_id": null,     // Use the default data-processing method
    "buffer_priority": "primary",   // Default column for m/z queries
    "sorting_rank": 0               // Sorted within entries; other arrays sort after
  },
  {
    "context": "spectrum",
    "path": "point.intensity",
    "data_type": "MS:1000521",
    "array_type": "MS:1000515",
    "array_name": "intensity array",
    "unit": "MS:1000131",           // Detector counts
    "buffer_format": "point",
    "transform": null,
    "data_processing_id": null,
    "buffer_priority": "primary",
    "sorting_rank": null           // Imposes no sorting order on the data
  }
]

```

This array index describes the table shown for the point layout. It is governed by the JSON Schema [schema/array_index.json](#).

9.2.1 Buffer format

Depending on the signal data layout in use, arrays are stored in different formats. The available `buffer_format` values are:

buffer_format	Used by	Meaning
point	point	The array is stored in the point layout. The point layout is all-or-nothing – every array must be point.
chunk_values	chunked	The “main” axis values bounded between a chunk’s start and end, encoded for better compressibility (in addition to Parquet’s own encoding).
chunk_start	chunked	The starting value of the main axis for the chunk, inclusive.
chunk_end	chunked	The ending value of the main axis for the chunk, inclusive. It should be less than the next chunk’s <code>chunk_start</code> .
chunk_encoding	chunked	A CURIE indicating how <code>chunk_values</code> was encoded.
chunk_secondary	chunked	Values of an array other than the main axis for the chunk.
chunk_transform	chunked	Raw bytes of an array in the chunk that was opaquely transformed (e.g. with MS-Numpress). May appear <i>in addition</i> to a referenced <code>chunk_values</code> or <code>chunk_secondary</code> column.

The chunked layout currently supports a single chunking dimension. A file in the chunked layout **MUST** use `chunk_start`, `chunk_end`, `chunk_encoding`, and `chunk_values` *exactly once* each.

9.2.2 Buffer priority and naming

In Parquet, all column names and types must be known before writing, and no two columns may share the same name and path. Normally there is one coordinate column (e.g. m/z or time) and one intensity column. If you have intensity arrays with different units or data types, they must be defined as separate arrays in the array index and therefore have distinct names. While uncommon for spectra, this is unavoidable for diagnostic traces stored as chromatograms.

For ergonomics, common columns should have simple, consistent names — this makes files easier to use from raw Parquet tooling. The most common version of each array type (as defined by the implementation) **SHOULD** have a `buffer_priority` of primary and receive a short, consistent name. The recommended short names:

Accession	Name	Column name
MS:1000514	m/z array	mz
MS:1000515	intensity array	intensity
MS:1000516	charge array	charge
MS:1000517	signal to noise array	signal_to_noise
MS:1000595	time array	time
MS:1000617	wavelength array	wavelength
MS:1002530	baseline array	baseline
MS:1002529	resolution array	resolution
MS:1002893	ion mobility array	ion_mobility
MS:1003007	raw ion mobility array	raw_ion_mobility
MS:1002816	mean ion mobility array	mean_ion_mobility
MS:1003154	deconvoluted ion mobility array	deconvoluted_ion_mobility
MS:1003008	raw inverse reduced ion mobility array	raw_inverse_reduced_ion_mobility
MS:1003006	mean inverse reduced ion mobility array	mean_inverse_reduced_ion_mobility
MS:1003155	deconvoluted inverse reduced ion mobility array	deconvoluted_inverse_reduced_ion_mobility
MS:1003153	raw ion mobility drift time array	raw_drift_time
MS:1002477	mean ion mobility drift time array	mean_drift_time
MS:1003156	deconvoluted ion mobility drift time array	deconvoluted_ion_mobility_drift_time

9.3 Data arrays, encoding, transformations, and Parquet

Parquet can write [page indices](#) on any *leaf* column, based on the value being stored *before* its [encoding](#) and [compression](#). This means we must take care when storing data cleverly: a transformation that obscures the stored value from the page index also disables predicate filtering on it. The techniques below are written in terms of spectra but apply more broadly.

9.3.1 Zero run stripping

Some vendors produce profile arrays with large “empty” regions of zero-intensity points along a semi-regular m/z axis. These regions hold little information, so all but the first and last zero-intensity points of a run are removed. This is only meaningful for profile data; readers **SHOULD** assume that zero runs have been stripped.

A **zero run** is a sequence of three or more zero values, reduced to just its first and last positions. Zero runs can be very long and, outside of scenarios that assume a complete coordinate grid, provide no value. If a zero run needs to be reconstructed beyond the flanking points, the same method used for filling null-marked values can extend the run.

EXAMPLE — Python — finding the positions that are *not* part of a zero run

```
def find_where_not_zero_run(data: Sequence[Number]) -> Sequence[int]:
    """
```

Construct a list of positions that are not part of a zero run.

A zero run is any position i such that:

- 1. `''x[i] == 0''`*
- 2. `''(i == 0) or (x[i - 1] == 0)''`*
- 3. `''(i == (len(x) - 1)) or (x[i + 1] == 0)''`*

We build a position list because we need to extract these positions from ALL dimension arrays for this entity, not just the current array.

```

n = len(data)
n1 = n - 1
was_zero = False
acc = []
i = 0
while i < n:
    v = data[i]
    if v is not None:
        if v == 0:
            if (was_zero or (len(acc) == 0)) and ((i < n1 and data[i + 1] ==
                ↪ 0) or i == n1):
                pass
            else:
                acc.append(i)
                was_zero = True
        else:
            acc.append(i)
            was_zero = False
    else:
        acc.append(i)
        was_zero = False
    i += 1
return np.array(acc, dtype=np.uintp)

```

9.3.2 Null marking

For spectra with many small gaps, even zero-run stripping leaves too much unhelpful information. Instead, we can replace the flanking zero-intensity points with null m/z and intensity values; Parquet then skips storing the expensive 32- and/or 64-bit values, retaining only the validity-bitmap bit. Separately, we fit a simple m/z-spacing model by weighted least squares of the form:

$$\delta m z \sim \beta_0 + \beta_1 m z + \beta_2 m z^2 + \epsilon$$

EXAMPLE — Python — fitting the weighted least-squares spacing model

```

class DeltaCurveRegressionModel:
    beta: np.ndarray

    def __init__(self, beta: np.ndarray):
        self.beta = beta

    @classmethod
    def fit(cls, mz_array, delta_array, weights=None, threshold=None, rank=2):
        if weights is None:
            weights = np.ones(len(mz_array))

        if threshold is None:
            threshold = 1.0

        # Drop all entries where the gap between m/z values > threshold
        raw = mz_array[1:][delta_array <= threshold]
        w = weights[1:][delta_array <= threshold]
        y = delta_array[delta_array <= threshold]

        # Build the design matrix

```

```

data = [np.ones_like(raw)]
for i in range(1, rank + 1):
    data.append(raw**i)
data = np.stack(data, axis=-1)

# Use the QR decomposition to solve the weighted least-squares problem
# to estimate weights predicting  $\square$  m/z.
# https://stats.stackexchange.com/a/490782/59613
chol_w = np.sqrt(w)
qr = np.linalg.qr(chol_w[:, None] * data)
v = qr.Q.T.dot(chol_w * y)
beta = solve_triangular(qr.R, v)

# Numerically equivalent to and more stable than the direct inversion
# beta = np.linalg.inv((data.T * w).dot(data)).dot(data.T * w).dot(y)
return cls(beta)

def predict(self, mz: float) -> float:
    acc = self.beta[0]
    for i in range(1, len(self.beta)):
        acc += self.beta[i] * mz ** i
    return acc

```

When reading null-marked data, use either the local second-median $\delta m/z$ or the learned model for that spectrum to compute the m/z spacing for singleton points, achieving a very accurate reconstruction. Because the non-zero m/z points are unchanged, a peak’s apex or centroid is unaffected. If a peak is composed of only three points — including the two zero-intensity flanks — no meaningful peak model can be fit anyway, so the minuscule angle change is effectively lossless.

The model parameters learned for each entry **MUST** be stored in that entry’s row in the associated meta-data table, as `mz_delta_model`.

QUESTION — Open item — generalise `mz_delta_model`

Should `mz_delta_model` become a CV parameter and be dissociated from m/z specifically, so the same mechanism can serve other coordinate axes? This is likely desirable.

All MS-Numpress compression methods remain available and still give superior size reduction, at the cost of slightly larger accuracy loss. Using Numpress is a *transformation* and requires the chunked layout.

9.3.2.1 Finding flanking zero pairs

Zero-intensity points on the flanks of peaks still cost non-trivial storage in sparse datasets. This step matches *only* the zero-intensity points that occur on the flanks of profile peaks — not all zero-intensity points. Once found, these indices construct the null mask (the Arrow “validity bitmap”), which is equivalent to how a Parquet column chunk would represent them.

EXAMPLE — Python — masking positions that are two zeros in a row

```

def is_zero_pair_mask(data: Sequence[Number]) -> "np.typing.NDArray[np.bool_]":
    '''Create a boolean mask for positions composed of two zeroes in a row.'''
    n = len(data)
    n1 = n - 1
    was_zero = False
    acc = []
    for i, v in enumerate(data):
        if v == 0:
            if was_zero or (i < n1 and data[i + 1] == 0):
                acc.append(True)
            else:
                acc.append(False)
            was_zero = True
        else:
            acc.append(False)
            was_zero = False
    return np.array(acc)

```

9.3.2.2 Decoding null pairs

Decoding null pairs — undoing null marking — means finding regions bounded by two nulls (or by the start of the array and a null, or a null and the end of the array), then filling the null values with either a locally estimated value (when more than one value is available to estimate the median delta) or the regression model described above (for a single point).

Unpaired null values **MAY** appear only as the first or last null in the array; any other unpaired null is an unrecoverable error. A run of three or more null values **MAY** be recoverable but should not occur under normal operation.

The locally estimated value **SHOULD** be the second median of the spacing of the current segment's non-null values. The regression model predicts the spacing from the single non-null value of a segment that has only one.

EXAMPLE — Python — filling *null*-marked values

```
def find_pairs(mask: Sequence[bool]) -> Sequence[int]:
    """
    Construct index ranges between pairs of ``True`` values in ``mask``.

    The first and last ranges include the start and end of the array even if
    the mask does not begin/end with a ``True``. The result has two columns:
    the start and end indices of each span between two ``True`` values (or the
    array termini).

    .. warning::
        This can fail or produce incorrect output if there are runs of ``True``
        values longer than 2. Buffer all data for an entry before calling this
        so that batches do not artificially disrupt pairs.
    """
    parts = []
    indices = np.where(mask)[0]
    if len(indices) == 0:
        return np.array([[0, len(mask)]])
    if indices[0] != 0:
        parts.append([0])
    parts.append(indices)
    if indices[-1] != len(mask) - 1:
        parts.append([len(mask) - 1])
    indices = np.concat(parts)
    indices = indices.reshape((-1, 2))
    indices[:, 1] += 1
    return indices

def estimate_median_delta(data: Sequence[Number]) -> tuple[Number,
    np.typing.NDArray]:
    """Find the 2nd median of ``np.diff(data)`` — a crude spacing estimate."""
    deltas = np.diff(data)
    median = np.median(deltas)
    deltas_below = deltas[deltas <= median]
    median = np.median(deltas_below)
    return median, deltas_below

def fill_nulls(data: pa.Array, common_delta: DeltaModelBase) ->
    np.typing.NDArray:
    """Fill ``null`` values using ``common_delta`` or a locally estimated median
    delta."""
    if not isinstance(common_delta, DeltaModelBase):
        if isinstance(common_delta, Number):
            common_delta = ConstantDeltaModel(common_delta)
        else:
            common_delta = DeltaCurveRegressionModel(np.asarray(common_delta))

    pair_indices = find_pairs(data.is_null())
```

```

chunks = []
for (start, end) in pair_indices:
    chunk = np.asarray(data.slice(start, end - start))
    n = len(chunk)
    has_real = chunk[~np.isnan(chunk)]
    n_has_real = len(has_real)
    if n_has_real == 1:
        # A singleton point with one or two sides to pad
        if n == 2:
            if np.isnan(chunk[0]):
                chunk[0] = chunk[1] - common_delta(chunk[1])
            else:
                chunk[1] = chunk[0] + common_delta(chunk[0])
        elif n == 3:
            dx = common_delta(chunk[1])
            chunk[0] = chunk[1] - dx
            chunk[2] = chunk[1] + dx
        else:
            raise Exception()
    else:
        # A run of values – estimate a more accurate delta from the data
        dx, _ = estimate_median_delta(has_real)
        if np.isnan(chunk[0]):
            chunk[0] = chunk[1] - dx
        if np.isnan(chunk[-1]):
            chunk[-1] = chunk[-2] + dx
    chunks.append(chunk)
return np.concat(chunks)

```

9.3.3 Null semantics for signal data

Unless otherwise noted, readers **SHOULD** treat null values in the sorting- rank-0 array of an entry as governed by this model, with parallel null values in any intensity arrays read as 0. The former should carry a transform of [MS:1003901](#) and the latter a transform of [MS:1003902](#). All other values at those points are read as-is, with null semantics meaning the value was absent. Writers using null marking **SHOULD** use null only for the first sorting dimension and its associated intensity value; all other columns should be written as-is.

9.4 Why a top-level node?

Couldn't we just unwrap the top-level point/chunk struct and move on?

Perhaps, but keeping a top-level node open three doors:

1. **Clear schema signalling.** When you see point at the root of the schema you know this is a point-layout file, not a chunked-layout one.
2. **Unaligned proprietary data.** A specialised writer or reader might embed information not directly connected to the primary schema's addressable unit (a spectrum, a data point). The top-level node leaves a place for that.
3. **Future table packing.** Early in mzPeak's design we tried to pack tables together as much as possible (as in the packed parallel tables layout), but that proved very inefficient to *write* despite being no slower to *read* – possibly an implementation detail rather than a Parquet limitation. Keeping a top-level node leaves the door open to revisit this without a schema-breaking change.

Chapter 10

Point Layout

The **point layout** stores data arrays as-is in parallel columns alongside a repeated index column. The top-level schema node is named `point` and is a group with an arbitrary number of columns. The entity index column **MUST** be the first column under `point`.

point		
spectrum_index	mz	intensity
1	213.2	1002
1	506.9	500
1	758	405
...	...	...
2	329.1	50
2	516.5	5002
2	783.8	302

This layout is simple, but carries several advantages:

- **Predicate filtering.** Scalar columns are easily filtered along the page-level range index, which makes multi-dimensional queries easy to write and optimise.
- **Transparent compression.** Arrays are encoded and compressed by Parquet, so the data is still stored compactly.

The trade-off: data **MUST** be stored as-is to keep the page index meaningful, so no additional obscuring transformations (delta encoding, Numpress, etc.) may be used. The zero-run stripping and null-marking methods remain available, because they only remove non-meaningful points from the array rather than transforming the values that remain.

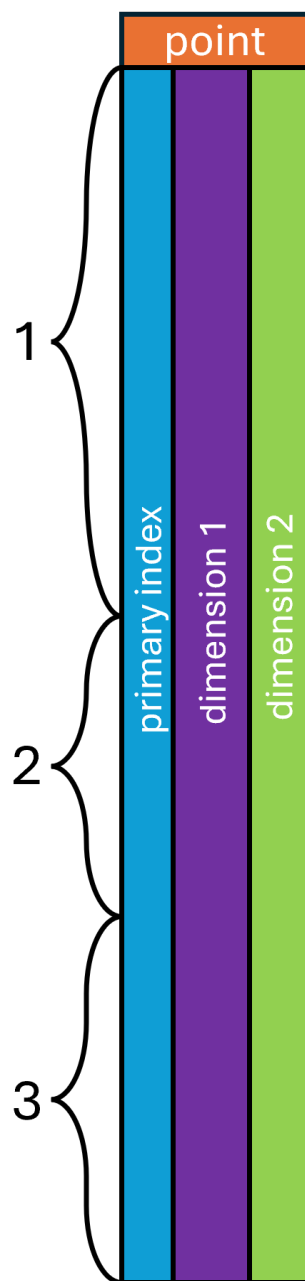


Figure 10.1: Point-layout schema: a single top-level point group holding a repeated spectrum_index column alongside parallel mz and intensity columns, with one table row per data point.

Chapter 11

Chunked Layout

The **chunked layout** treats one array — which must be sorted — as the “main” axis, cutting it into chunks of a fixed size along that coordinate space (for example steps of 50 m/z) and taking the same segments from the parallel arrays. The main axis chunks’ start, end, and a repeated index are recorded as columns; each array may then be stored as-is or with an opaque transform (delta encoding, Numpress, ...). The start/end interval permits granular random access along *both* the main axis and the source index. The top-level schema node is named `chunk`, and the entity index column **MUST** be its first column.

chunk spectrum_index	mz_chunk_start	mz_chunk_end	mz_chunk_values	chunk_encoding	intensity
1	200	250	[0.0013, ..., 0.0013]	MS:1003089	[...]
1	250	300	[0.0014, ..., 0.0014]	MS:1003089	[...]
1	500	550	[0.0014, ..., 0.0015]	MS:1003089	[...]
...	...	...	...	...	...
2	200	250	[0.0013, ..., 0.0013]	MS:1003089	[...]
2	350	400	[0.0014, ..., 0.0014]	MS:1003089	[...]
2	400	450	[0.0013, ..., 0.0014]	MS:1003089	[...]

This example uses delta encoding for the m/z chunk values, which is reconstructed with very high precision for 64-bit floats. The m/z values inside `mz_chunk_values` are not visible to the page index, but the `_chunk_start` and `_chunk_end` columns are. The chunk values remain subject to Parquet encodings, so they can be byte-shuffled for further compression.

11.1 Column naming rules

1. **<entity>_index** (integer) — the index key for the entity this chunk belongs to.
2. **<array_name>_chunk_start** (float64) — the first coordinate value in the chunk, inclusive. Its array-index buffer_format **MUST** be `chunk_start`.
3. **<array_name>_chunk_end** (float64) — the last coordinate value in the chunk, inclusive. Its array-index buffer_format **MUST** be `chunk_end`.
4. **<array_name>_chunk_values** (list) — the encoded coordinates from `array_name` according to `chunk_encoding`. Its array-index buffer_format **MUST** be `chunk_values`.
5. **chunk_encoding** (CURIE) — how `<array_name>_chunk_values` were encoded; see Chunk encodings. Its array-index buffer_format **MUST** be `chunk_encoding`.

All other columns are expected to be list arrays whose names are simply their `array_name` (with buffer_format `chunk_secondary`), or surrogate arrays with buffer_format `chunk_transform`.

11.2 Splitting data into chunks

A chunk table may be partitioned in any pattern, as long as the chunks are non-overlapping and ascending. The chunking procedure must be null-aware — in particular, aware of the null pairs that mark masked regions. The granularity is configurable, trading random-access granularity against compression efficiency.

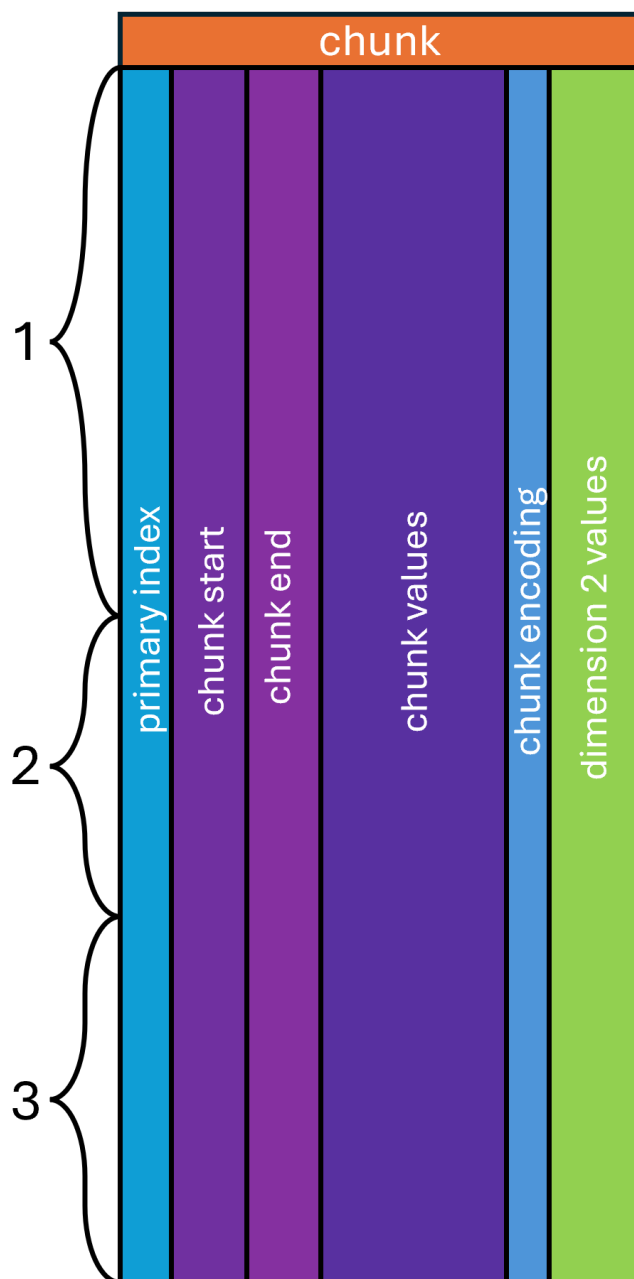


Figure 11.1: Chunked-layout schema: a top-level chunk group holding `spectrum_index`, `mz_chunk_start` and `mz_chunk_end` bounds, an encoded `mz_chunk_values` list, a `chunk_encoding` column, and an intensity list, with one table row per chunk.

EXAMPLE – Python – partitioning into chunks of width k with null pairs present

```

import pyarrow as pa

def null_chunk_every(data: pa.Array, width: float) -> list[tuple[int, int]]:
    """
    Partition a sorted numerical array into segments spanning `width` units.
    This operation is null-aware, so sparse arrays can be partitioned.
    Returns the (start, end) index of each chunk.
    """
    start = None
    n = len(data)
    i = 0
    # Find the first non-null position
    while i < n:
        v = data[i]
        if v.is_valid:
            start = v.as_py()
            break
        else:
            i += 1

    # If we never found a non-null position, just return a single chunk
    if start is None:
        return [(0, n)]

    chunks = []
    offset = 0
    threshold = start + width
    i = 0
    while i < n:
        v = data[i]
        if v.is_valid:
            v = v.as_py()
            if v > threshold:
                if ((i + 1) < n) and (not data[i + 1].is_valid):
                    while ((i + 1) < n) and (not data[i + 1].is_valid):
                        i += 1
                    # Avoid a chunk of length 1, especially a null point; if so,
                    # relax the width requirement.
                    if i - offset > 1:
                        chunks.append((offset, i))
                        offset = i
                    while threshold < v:
                        threshold += width
                # Look ahead: this value is null, but the next is not.
                elif ((i + 1) < n) and (data[i + 1].is_valid):
                    i += 1
                    v = data[i].as_py()
                    if v > threshold:
                        i -= 1
                        chunks.append((offset, i))
                        offset = i
                        while threshold < v:
                            threshold += width
                    i += 1
            else:
                i += 1
    if offset != n:
        chunks.append((offset, n))
    return chunks

```

11.3 Chunk encodings

11.3.1 Basic encoding

Chunk-encoding CV term: [MS:1000576 – no compression](#)

When storing centroids, or data that are not similarly spaced (as is usually the case for pre-centroided spectra), but still wanting the chunked layout, no special encoding of the chunk values is necessary. Values within each chunk are written as-is to the chunk-values array. This does not improve compressibility, but it keeps a consistent schema for other entries that *would* benefit from a different encoding.

NOTE

The start point is *excluded* from the chunk-values array.

11.3.2 Delta encoding

Chunk-encoding CV term: [MS:1003089 – truncation, delta prediction and zlib compression](#)

When data lie on a locally (*almost*) uniform grid using 64-bit floats, compression improves by computing a delta encoding of the coordinates.

NOTE

The start point is *excluded* from the chunk-values array.

EXAMPLE – Python – null-aware delta encode/decode

```
import pyarrow as pa

def null_delta_encode(data: pa.Array) -> pa.Array:
    """
    Delta-encode an Arrow array containing nulls. Nulls are encoded as null
    values and treated as 0.0 for the purpose of computing the next delta.
    """
    acc = []
    it = iter(data)
    # The first entry is the point of reference but not part of the delta
    # sequence unless it is 'null'.
    last = next(it)
    if not last.is_valid:
        acc.append(last)

    for item in it:
        if item.is_valid:
            val = item.as_py()
            if last.is_valid:
                acc.append(pa.scalar(val - last.as_py()))
            else:
                # treat the last value as 0.0
                acc.append(item)
            last = item
        else:
            acc.append(item)
            # carry the null forward
            last = item
    return pa.array(acc)

def null_delta_decode(data: pa.Array, start: pa.Scalar) -> pa.Array:
    """Decode an Arrow array that was delta-encoded *with* nulls."""
    acc = []
    if not data[0].is_valid:
        if not data[1].is_valid:
            # started at a non-null value immediately followed by a null pair
            acc.append(start)
            start = pa.scalar(None, data.type)
        else:
            acc.append(start.as_py())
    last = start
    for item in data:
        if item.is_valid:
            val = item.as_py()
            if last.is_valid:
                last = pa.scalar(val + last.as_py())
            acc.append(last)
        else:
            # last value assumed zero
```

```

        acc.append(item)
        last = item
    else:
        acc.append(item)
        last = item
    return pa.array(acc)

```

11.3.3 Numpress linear encoding

Chunk-encoding CV term: [MS:1002312](#) – MS-Numpress linear prediction compression

This uses the MS-Numpress linear-prediction method ([10.1074/mcp.O114.037879](#)) to compress the chunk values as raw bytes. Numpress produces a buffer of an 8-byte fixed point, a 4-byte value 0, a 4-byte value 1, and then 2-byte residuals for all subsequent values. The array is therefore, by definition, not alignable to a 4- or 8-byte type. It also has no concept of nullity, which makes it **incompatible with null marking**.

To store Numpress-linear-encoded arrays, an extra column `<array_name>_numpress_linear_bytes` is added alongside the `<array_name>_chunk_values` column. It is a list of byte arrays (`large_list<u8>` in Arrow parlance – **not** `large_binary`; see the [discussion of string-type “optimisation”](#)). Its array-index entry **MUST** have `buffer_format` `chunk_transform` and the same array type, array name, data type, unit, and data-processing ID as the `_chunk_values` column. The transform field **MUST** be `MS:1002312`.

NOTE

The start point is *included* in the chunk-values array – it is a specific component of the Numpress-encoded bytes.

11.4 Opaque array transforms

Sometimes we prefer to store data lossily in non-uniform, unaligned, or otherwise non-standard types that have no physical representation in Parquet. MS-Numpress’s short logged float (SLOF) and positive-integer encodings are good examples. While Numpress-linear handles the coordinate dimension, opaque transforms can also encode the *secondary* arrays in chunks. These columns are recorded in the array index with `buffer_format` `chunk_transform`, and the transform field is the CURIE for the relevant method – for example [MS:1002314](#) for MS-Numpress SLOF. The column’s physical type **MUST** be a list of byte arrays, though the type in the array index **MUST** be the *decoded* array’s real type. Column names **SHOULD** be of the form `<array_name>_<transform_name>_bytes`, e.g. `intensity_numpress_slof_bytes`.

11.5 Reading a single entry from the chunked encoding

To read a single entry (spectrum, chromatogram, ...) stored in chunks:

0. Identify which columns are annotated as `chunk_start`, `chunk_end`, `chunk_encoding`, and `chunk_values` in the array index. The `<entity_type>_index` column **MUST** be the first column in the table, so it always has index 0.
1. Find the row group containing the entry’s index value via the row-group metadata. Optionally, if the page index is available, find the row ranges of the pages that contain that index.
2. Read the selected row group (or page row range) and filter to rows whose `<entity_type>_index` equals the entry’s index.
3. Optionally, sort the rows by their `chunk_start` value – usually ascending for the quantity being measured.
4. Process each selected row, decoding its `chunk_values` according to `chunk_encoding` and any transform in the array index. Unpack the `chunk_secondary` columns and apply any transforms, accumulating each column’s arrays across rows. Some transforms require additional information from the entity’s metadata table.
5. If the entry has additional auxiliary arrays, read and decode them from the metadata table.

Chapter 12

Auxiliary Data Arrays

When an array is present in an entry but is **not** encoded as a column in the schema, it must be stored as an **auxiliary array**. This happens when mixing different kinds of detectors in a single collection, and especially with diagnostic traces, where every array might have a different length along a shared time axis, or be sub-sampled.

Auxiliary data arrays have a schema similar to mzML's [binaryDataArray](#), encoded in Parquet. They are governed by the JSON Schema [schema/auxiliary_array.json](#).

```
optional group auxiliary_arrays (List) {
  repeated group list {
    optional group item {
      optional group data (List) {
        repeated group list {
          required int32 item (Int(bitWidth=8, isSigned=false));
        }
      }
      optional group name {
        optional group value {
          optional int64 integer;
          optional double float;
          optional binary string (String);
          optional boolean boolean;
        }
        optional binary accession (String);
        optional binary name (String);
        optional binary unit (String);
      }
      optional binary data_type (String);
      optional binary compression (String);
      optional binary unit (String);
      optional group parameters (List) {
        repeated group list {
          optional group item {
            optional group value {
              optional int64 integer;
              optional double float;
              optional binary string (String);
              optional boolean boolean;
            }
            optional binary accession (String);
            optional binary name (String);
            optional binary unit (String);
          }
        }
      }
    }
  }
}
optional binary data_processing_ref (String);
}
```

WARNING — Auxiliary arrays cannot be sliced

Because an auxiliary array is stored as an opaque encoded buffer rather than a first-class column, it **cannot** be searched or sliced without decoding the whole array — exactly as in mzML. The associated metadata row records the count in `number_of_auxiliary_arrays`, so a reader can cheaply decide whether the more expensive decoding step is worthwhile before attempting it.

Chapter 13

The Index File — `mzpeak_index.json`

An mzPeak archive is made up of multiple named files. To leave room for future files and avoid complicated file-name resolution, an **index file** identifies the contents of each file and broadly defines the kind of schema it carries. The file **MUST** be serialised as UTF-8.

```
{
  "files": [
    {
      "name": "spectra_data.parquet",
      "entity_type": "spectrum",
      "data_kind": "data arrays"
    },
    {
      "name": "spectra_metadata.parquet",
      "entity_type": "spectrum",
      "data_kind": "metadata"
    },
    {
      "name": "chromatograms_data.parquet",
      "entity_type": "chromatogram",
      "data_kind": "data arrays"
    },
    {
      "name": "chromatograms_metadata.parquet",
      "entity_type": "chromatogram",
      "data_kind": "metadata"
    }
  ],
  "metadata": {
    "version": "0.9.0",
    "cv_list": [
      {
        "id": "MS",
        "full_name": "Proteomics Standards Initiative Mass Spectrometry Ontology",
        "uri": "http://purl.obolibrary.org/obo/ms/4.1.248/ms.obo",
        "version": "4.1.248"
      },
      {
        "id": "UO",
        "full_name": "Units of measurement ontology",
        "uri": "http://purl.obolibrary.org/obo/uo/releases/2026-01-16/uo.obo",
        "version": "2026-01-16"
      }
    ],
    "file_description": { }
  }
}
```

The metadata object carries the archive version, the `cv_list` declaring every controlled vocabulary used (with source URI and version, so CURIEs resolve reproducibly), and the file-level metadata objects.

Governed by the JSON Schema [schema/mzpeak_index.json](#).

Each entry pairs a `data_kind` with an `entity_type`. Both are *loose enumerations* expected to grow over time; resolving files by these controlled terms is more robust than matching file names.

13.1 File-level metadata

File-level metadata **SHOULD** be stored in `mzpeak_index.metadata` and in the metadata Parquet files' key-value pairs, as JSON encoded according to the schemas below:

- `cv_list`
- `file_description`
- `instrument_configuration_list`
- `data_processing_method_list`
- `software_list`
- `sample_list`
- `scan_settings_list`
- `run`

QUESTION – Open item – cleartext vs. encryptable metadata

Anything in `mzpeak_index.json` is necessarily cleartext to all readers unless ZIP encryption is used – and ZIP encryption is known to be flawed and inconsistent. Anything in a Parquet footer's key-value pairs *is* encryptable. The index is JSON for convenience and ease of access from scripting languages; whether some fields should move to encryptable Parquet metadata is unresolved.

Chapter 14

Data Kinds

The `data_kind` field tells the reader the *semantics* of the data in a file, and roughly what schema to expect. There are currently five controlled values:

data_kind	Expected layout	Meaning
data_arrays	point or chunked	Signal data, usually in its “raw” form, for the file’s <code>entity_type</code> .
peaks	point or chunked	Like <code>data_arrays</code> , but <i>processed</i> — implies a less-refined entry exists in a <code>data_arrays</code> file. This is how profile and centroid signal coexist for a spectrum.
metadata	packed parallel table	Everything but the homogeneous signal arrays. May still be large.
proprietary	implementation-defined	Entirely the purview of the writer (often an instrument vendor). May not be Parquet. Should be ignored unless the reader is for that vendor.
other	implementation-defined	None of the above. May not be Parquet.

Any value outside this list is treated as `other`. Files marked `proprietary` and `other` are implementation-defined, but `other` files may still be of interest to non-vendor readers (for example, text or XML configuration files in an evolving metadata landscape). Vendors are encouraged to use `proprietary` for binary or hard-to-digest contents.

14.1 Adding a new data kind

This list is necessarily incomplete — new use-cases will emerge. For example, one might store extracted LC-(IM)-MS feature bounding boxes as a separate file. To add a new data kind:

1. **Pick a name** that fits in the index JSON. Prefer lower-case — e.g. `feature_map` for extracted features.
2. **Pick a layout** (or layouts) for the data kind — e.g. the packed parallel table for lists of bounding boxes with associated metadata.
3. **Describe the relationships** with valid entity types. Prefer simple relationships (one-to-one, one-to-many). If no existing entity type fits, create a new one — an LC-MS feature might associate with spectrum, but there is no clean one-to-one or one-to-many relationship between spectra and LC-MS features, so a new entity type may be needed.

Chapter 15

Entity Types

The `entity_type` tells the reader what is *being* described in a file and, in concert with the `data_kind`, helps connect the right file to the right reader API. There are currently four controlled values:

entity_type	Description
spectrum	Mass spectra — entities occurring at a single point in time (or as close to that as framed or cycled acquisition allows), with a mass-related coordinate such as m/z or neutral mass.
chromatogram	Measurements <i>over time</i> — chromatograms and, for now, diagnostic traces (see below).
wavelength spectrum	Like spectrum, but the coordinate is an electromagnetic wavelength. Analyzers measuring wavelength are far more heterogeneous than mass analyzers. Time series over wavelength may instead be stored as chromatogram entries.
other	None of the above — something not yet covered by this living specification.

Any value outside this list is treated as other.

15.1 Decided at HUPO-PSI 2026 (Rome)

The May 2026 working session reached consensus on several additions. These are recorded here as direction for upcoming drafts; their concrete schemas are still being specified.

SUCCESS — Diagnostic traces become their own namespace

Instrument **diagnostic traces** (pump pressure, source current, temperature, voltages, flow rate, ...) will be stored under a **new entity-type namespace** (working name: *diagnostic traces*), distinct from chromatogram. They share the time-axis machinery of chromatograms but are semantically different measurements, and separating them keeps chromatogram queries clean. Until the dedicated namespace is specified, such traces may be carried as chromatogram entries with appropriate CV typing.

INFO — Ion mobilograms — likely a future facet

Ion **mobilograms** (ion mobility as the primary axis) may be formalised as a separate facet. This was judged not yet required, but the format leaves room for it.

INFO — Imaging MS and regions of interest

Imaging MS is currently handled via pixel coordinates in the spectrum metadata table. **Regions of interest** can be layered on top as spatial-annotation polygons (for example, feature-

extraction bounding boxes). Parquet was chosen over ZARR for long-term stability and cross-language support.

INFO – Intelligent data-acquisition traces

Traceability of an instrument's **decision-making during acquisition** (e.g. intelligent/IAPI-driven acquisition) is a recognised future need. It can be encoded as CV parameters per scan; cross-instrument interoperability will require dedicated ontology work.

15.2 Adding a new entity type

When a genuinely new kind of measured entity appears that does not fit spectrum, chromatogram, or wavelength spectrum, a new entity type may be introduced. Prefer a short, lower-case name, define its primary coordinate axis and the data kinds it supports, and describe its relationship to existing entity types.

QUESTION – Open item – naming

Should the chromatogram entity type be renamed to something broader such as *traces*, now that diagnostic traces are moving to their own namespace? Left open.

Chapter 16

Spectra

Spectra are described by up to three files: a **signal** file (`spectra_data.parquet`), an optional **peaks** file (`spectra_peaks.parquet`), and a **metadata** file (`spectra_metadata.parquet`).

ABSTRACT — Profile vs. centroid — where each goes

By consensus at HUPO-PSI 2026, **profile** data goes in `spectra_data.parquet` and **centroid** data goes in `spectra_peaks.parquet`, *always*. When a file contains both for the same spectrum, both files are present and the metadata row carries both `MS_1003060_number_of_data_points` and `MS_1003059_number_of_peaks` so a reader knows which file(s) to read. A reader exposes a mode flag (profile / centroid) indicating which representation the caller wants.

For TimsTOF-style data that is centroided in m/z but profiled in ion mobility, the consensus is to treat it as centroid for the mass-spectrum dimension and place it in `spectra_peaks.parquet`.

16.1 Spectrum signal data — `spectra_data.parquet`

```
{ "name": "spectra_data.parquet", "entity_type": "spectrum", "data_kind": "data arrays" }
```

The spectrum signal data is encoded using either point layout or chunked layout. The entity index column **MUST** be named `spectrum_index`, and a co-located time column, if written, **SHOULD** be named `spectrum_time`. Non-mass spectra (UV, DAD) belong in `wavelength_spectra_data.parquet`.

When using null marking, follow the null semantics for signal data carefully for profile data.

WARNING — Profile only

Only **profile** spectra are written here. Centroid spectra — including centroided views of profile spectra when both modes are stored — **MUST** instead be written to `spectra_peaks.parquet`. The number of points written here for a spectrum **MUST** be recorded in the `MS_1003060_number_of_data_points` column of `spectra_metadata.parquet`, to support read planning.

16.1.1 Recommended Parquet encodings

Column	Encoding
<code>spectrum_index</code>	delta encoding — ideal for repetitive or slowly increasing integers.
<code>spectrum_time</code>	byte stream split
<code>m/z arrays</code>	byte stream split (byte shuffling), or RLE dictionary when there is ion-mobility data.
<code>ion-mobility arrays</code>	RLE dictionary ; byte shuffling tends not to help. Consider increasing the dictionary page size.

16.2 Spectrum peak data – spectra_peaks.parquet

```
{ "name": "spectra_peaks.parquet", "entity_type": "spectrum", "data_kind": "peaks" }
```

The spectrum peak lists, stored separately from the raw signal in spectra_data.parquet. The entity index column **MUST** be named spectrum_index, and a co-located time column, if written, **SHOULD** be named spectrum_time. Any centroid spectra **MUST** be written here, not to spectra_data.parquet. The number of peaks written for a spectrum **MUST** be recorded in the MS_1003059_number_of_peaks column of spectra_metadata.parquet, to support read planning.

16.3 Spectrum metadata – spectra_metadata.parquet

```
{ "name": "spectra_metadata.parquet", "entity_type": "spectrum", "data_kind": "metadata" }
```

This table uses the packed parallel metadata table schema. Column order is generally unspecified, but spectrum.index, scan.source_index, precursor.source_index, and selected_ion.source_index **MUST** be the first column of their respective facets. Where the lists below say **MAY**, that value may be stored either as a column or as an entry in the parameters list – a column usually makes more sense when the value is usually present.

16.3.1 spectrum(group)

- **index** (uint64) – the ascending 0-based index. **MUST** increment by 1 per entry and **SHOULD** be written in time-sorted ascending order. This is the primary key for the spectrum facet and the root unit of addressability.
- **id** (string) – the “nativeID” string identifier, formatted per a [native identifier format](#). The specific format **SHOULD** be given in the file-level metadata under file_description.source_files[0].parameters, as in mzML.
- **time** (float64) – the data-acquisition start time. **SHOULD** be replicated from the parallel scan facet for simpler filtering; for a spectrum with multiple scans it **SHOULD** be the minimum value if the run is in acquisition-time order.
- **MS_1000511_ms_level** (integer) – the MS stage number, or null for non-mass spectra.
- **data_processing_ref** (string) – the identifier of a data_processing that governs this spectrum if it deviates from the default in run.default_data_processing_id; null otherwise.
- **parameters** (list) – controlled or uncontrolled parameters; see the parameters list.
- **number_of_auxiliary_arrays** (integer) – the count of auxiliary arrays in this row’s auxiliary_arrays column; lets a reader cheaply decide whether to decode them.
- **auxiliary_arrays** (list) – structures describing arrays that did not fit the arrays-and-columns constraints. These may be large; load eagerly with care.
- **mz_delta_model** (list of float64) – parameters of the m/z delta model used to reconstruct null-marked data.
- **MS_1000525_spectrum_representation** (CURIE) – e.g. MS:1000128 “profile spectrum” or MS:1000127 “centroid spectrum”.
- **MS_1000465_scan_polarity** (integer) – 1 (positive), -1 (negative), or null.
- **MS_1000559_spectrum_type** (CURIE) – a child of MS:1000559, e.g. MS1 spectrum (MS:1000579), MSn spectrum (MS:1000580).
- **MS_1003060_number_of_data_points** (integer) – profile points stored in spectra_data.parquet.
- **MS_1003059_number_of_peaks** (integer) – discrete peaks stored in spectra_peaks.parquet.
- **MAY** supply a child of MS:1003058 (spectrum property) one or more times – e.g. base peak m/z, total ion current.
- **MAY** supply a child of MS:1000499 (spectrum attribute) one or more times – e.g. MS_1000796_spectrum_title.

16.3.2 scan(group)

A scan or acquisition from the original raw file used to create a spectrum.

- **source_index** (uint64) – the index of the spectrum this scan belongs to (foreign key).

- **scan_index** (uint64) — the ascending 0-based index, incrementing by 1 per entry; uniquely identifies a scan, especially with multiple scans per spectrum (summing/averaging).
- **spectrum_reference** (string) — another spectrum corresponding to this scan. For local spectra, its id; for *external* sources, a **USI SHOULD** be used. For unpublished collections, use USI000000 as the collection identifier with the id of a source file in `file_description.source_files`.
- **instrument_configuration_ref** (integer) — the `instrument_configuration` governing this scan.
- **parameters** (list).
- **ion_mobility_value** (float64) — optional ion-mobility measurement for this scan.
- **ion_mobility_type** (CURIE) — optional; a child of [MS:1002892](#).
- **scan_windows**.
- **MAY** supply children of [MS:1000503](#) (scan attribute), [MS:1000018](#) (scan direction, once), and [MS:1000019](#) (scan law, once).

16.3.3 precursor(group)

The method of precursor-ion selection and activation.

- **source_index** (uint64) — the spectrum this precursor belongs to (foreign key).
- **precursor_index** (uint64) — the spectrum the precursor was created from (foreign key).
- **precursor_id** (string) — the id of the spectrum referenced by `precursor_index`.
- **isolation_window** (group) — the isolation/selection window.
 - **parameters** (list).
 - **MUST** supply children of [MS:1000792](#) (isolation-window attribute) one or more times; promote to columns when available — e.g. isolation-window target m/z, lower offset, upper offset.
- **activation** (group) — the activation/dissociation type and energy.
 - **parameters** (list).
 - **MAY** supply children of [MS:1000510](#) (precursor activation attribute).
 - **MUST** supply [MS:1000044](#) (dissociation method) or a child, one or more times.

16.3.4 selected_ion(group)

An ion isolated for dissociation.

- **source_index** (uint64) — the spectrum this selected ion belongs to (foreign key).
- **precursor_index** (uint64) — the spectrum the selected ion was created from (foreign key).
- **ion_mobility_value** (float64) / **ion_mobility_type** (CURIE, child of [MS:1002892](#)) — optional.
- **parameters** (list).
- **MUST** supply a child of [MS:1000455](#) (ion selection attribute) one or more times — e.g. selected-ion m/z, charge state, intensity.

QUESTION — Open item — generic ion-mobility storage

Is there a better way to make ion-mobility storage generic over its type (“ion mobility drift time”, “inverse reduced ion mobility”, “FAIMS compensation voltage”)? Left open.

Chapter 17

Chromatograms

Chromatograms are described by a **signal** file (`chromatograms_data.parquet`) and a **metadata** file (`chromatograms_metadata.parquet`). Diagnostic traces are currently carried here as well, pending their dedicated entity-type namespace.

17.1 Chromatogram signal data — `chromatograms_data.parquet`

```
{ "name": "chromatograms_data.parquet", "entity_type": "chromatogram", "data_kind": "data"
  ↳ "arrays" }
```

The signal data is encoded using either point layout or chunked layout. The entity index column **MUST** be named `chromatogram_index`. The default primary axis is a `MS:1000595 time array`, though the unit is up to the writer.

17.1.1 Recommended Parquet encodings

Column	Encoding
<code>chromatogram_index</code>	delta encoding
<code>time arrays</code>	byte stream split (byte shuffling)

17.2 Chromatogram metadata — `chromatograms_metadata.parquet`

```
{ "name": "chromatograms_metadata.parquet", "entity_type": "chromatogram", "data_kind":
  ↳ "metadata" }
```

This table uses the packed parallel metadata table schema.

17.2.1 `chromatogram` (group)

- **index** (integer) — the ascending 0-based index, incrementing by 1 per entry and **SHOULD** be time-sorted ascending. Primary key for the chromatogram facet.
- **id** (string) — a unique string identifier.
- **MS_1000465_scan_polarity** (integer) — 1, -1, or null.
- **MS_1000626_chromatogram_type** (CURIE) — e.g. total ion current (`MS:1000235`), selected ion current (`MS:1000627`), base peak (`MS:1000628`), absorption (`MS:1000812`).
- **data_processing_ref** (integer) — a `data_processing` governing this chromatogram if it deviates from `run.default_data_processing_id`; null otherwise.
- **parameters** (list).
- **number_of_auxiliary_arrays** (integer) and **auxiliary_arrays** (list) — see auxiliary data arrays.

- **MS:1003060_number_of_data_points** (integer) — data points stored in chromatograms_data.parquet.
- **MAY** supply a child of **MS:1000808** (chromatogram attribute) one or more times.

17.2.2 precursor(group)

The method of precursor-ion selection and activation.

- **source_index** (integer) — the chromatogram this precursor belongs to (foreign key).
- **precursor_index** (integer) — the chromatogram the precursor was created from (foreign key).
- **precursor_id** (string) — the id of the chromatogram referenced by precursor_index.
- **isolation_window** (group) — as for spectra: **MUST** supply children of **MS:1000792**.
- **activation** (group) — as for spectra: **MUST** supply **MS:1000044** (dissociation method) or a child.

17.2.3 selected_ion(group)

- **source_index** (integer) / **precursor_index** (integer) — foreign keys.
- **ion_mobility_value** (float) / **ion_mobility_type** (CURIE) — optional.
- **parameters** (list).
- **MUST** supply a child of **MS:1000455** (ion selection attribute) one or more times — selected-ion m/z, charge state, intensity.

Chapter 18

Wavelength Spectra

Wavelength spectra — UV, DAD, and other electromagnetic-radiation (EMR) spectra — are stored independently from mass spectra so the two modalities can have divergent schemas without inflating the number of empty columns, and so a reader need not sift through mass spectra to find EMR spectra (or vice versa). These files **SHOULD** be present only if wavelength spectra are included in the archive.

18.1 Wavelength spectrum signal data — wavelength_spectra_data.parquet

```
{ "name": "wavelength_spectra_data.parquet", "entity_type": "wavelength spectrum",  
  ↪ "data_kind": "data arrays" }
```

The signal data is encoded using either point layout or chunked layout. The entity index column **MUST** be named `wavelength_spectrum_index`, and a co-located time column, if written, **SHOULD** be named `wavelength_spectrum_time`.

When using null marking, follow the null semantics for signal data carefully for profile data.

18.2 Wavelength spectrum metadata — wavelength_spectra_metadata.parquet

```
{ "name": "wavelength_spectra_metadata.parquet", "entity_type": "wavelength spectrum",  
  ↪ "data_kind": "metadata" }
```

This table uses the packed parallel metadata table schema. It mirrors the spectrum metadata layout but omits the precursor and selected_ion facets, because EMR spectra have not been observed with isolation and fragmentation. `spectrum.index` and `scan.source_index` **MUST** be the first column of their respective facets.

18.2.1 spectrum(group)

- **index** (uint64) — ascending 0-based index, incrementing by 1 per entry, **SHOULD** be time-sorted. Primary key.
- **id** (string) — the “nativeID” string, per a [native identifier format](#).
- **time** (float64) — data-acquisition start time; **SHOULD** mirror the parallel scan facet.
- **data_processing_ref** (string) — a data_processing governing this spectrum if it deviates from the default; null otherwise.
- **parameters** (list).
- **number_of_auxiliary_arrays** (integer) / **auxiliary_arrays** (list) — see auxiliary data arrays.
- **MS_1000559_spectrum_type** (CURIE) — e.g. **MS:1000804** “electromagnetic radiation spectrum”.
- **MS_1003060_number_of_data_points** (integer) — profile points stored in `wavelength_spectra_data.parquet`.

- **MAY** supply a child of [MS:1003058](#) (spectrum property) one or more times — e.g. λ_{max} , lowest observed wavelength, total ion current.
- **MAY** supply a child of [MS:1000499](#) (spectrum attribute) one or more times — e.g. [MS_1000796_-spectrum_title](#).

18.2.2 scan (group)

- **source_index** (uint64) — the spectrum this scan belongs to (foreign key).
- **instrument_configuration_ref** (integer) — the instrument_configuration governing this scan.
- **parameters** (list).
- **scan_windows** — see the equivalent substructure for spectra.
- **MAY** supply a child of [MS:1000503](#) (scan attribute) one or more times.

Chapter 19

Implementations

mzPeak is backed by five independent, from-scratch implementations — not bindings to a single core. Building on the widely available [Apache Parquet](#) and [Apache Arrow](#) libraries keeps the on-disk structure language-independent, so each implementation reads and writes the same archives natively.

Language	Capability	API style
Rust	read / write	The reference reader and writer.
Python	read	Zero-copy Arrow / Pandas.
R	read	dplyr-compatible table access.
C#	read / write	Includes a Thermo RawFileReader demo.
JavaScript / TypeScript	read	Runs in the browser, Node, and Deno.

19.1 Rust — reference implementation

[HUPO-PSI/mzPeak](#) (mzpeak_prototyping) is the reference reader and writer. As the canonical implementation, it tracks the specification most closely and is the place to confirm intended behaviour when the prose is ambiguous.

19.2 Python

A read-only library exposing mzPeak data as zero-copy [Apache Arrow](#) tables and [Pandas](#) DataFrames, so metadata and signal can be queried with the analytical tooling proteomics and metabolomics users already know.

19.3 R

A read-only library offering dplyr-compatible access to the packed metadata and signal tables, developed in coordination with the [R for Mass Spectrometry](#) community. The current interface uses S6-style classes; an S4 interface (for full Bioconductor ecosystem compatibility) is planned.

19.4 C

[HUPO-PSI/mzPeak.NET](#) is a read/write implementation for the .NET ecosystem. It ships with a demonstration that reads vendor data through Thermo's RawFileReader and writes it to mzPeak.

19.5 JavaScript / TypeScript

A read-only implementation that runs in the browser, Node.js, and Deno, making mzPeak archives directly inspectable on the web. An [online viewer](#) demonstrates reading local and remote files — including extracted-ion chromatograms, base-peak chromatograms, and metadata inspection — entirely client-side.

NOTE — A common cross-language API is in progress

A language-agnostic, OpenAPI-style description of the shared reader/writer interface (open, get_spectrum, get_chromatogram, iterate, slice, and related methods) is planned, to keep the implementations aligned as the format matures.

TIP — Building your own?

The format is language-independent — start from this specification and validate your output against the conformance validator (see Tools). Contributions and feedback are welcome via the [HUPO-PSI repositories](#).

Chapter 20

Tools

The software ecosystem around mzPeak is open source. This page lists the utilities built on top of the format; for the read/write libraries themselves, see Implementations.

20.1 Converters

- **mzML2mzPeak** — converts imzML / mzML to mzPeak (and back), with full round-trip verification. It reads input through [mzdata](#).
- *Roadmap*: a direct **vendor RAW** → **mzPeak** path (for example inside ProteoWizard [msconvert](#)) so every vendor format converts in a single step, embedding the original acquisition method as provenance.

20.2 Validation

- **mzPeakValidator** — a language-independent, profile-driven conformance validator (`mzpeak-validate file.mzpeak`). It checks index-file structure, schema consistency, JSON-schema compliance, CV accession/name agreement, reasonable data types, and cross-file consistency (for example that the declared number of data points matches the signal table).

20.3 Viewers

- **mzPeak Explorer** — opens any .mzpeak file directly in the browser, streamed over HTTP range requests with no download. Built on the [TypeScript implementation](#).
- **mzPeakIV** — an imaging viewer for mass-spectrometry imaging (MSI) datasets. It streams a .mzpeak file in place and can assign masses to RGB channels to reconstruct tissue anatomy directly in the browser.

20.4 Example data

- [Example data corpus](#) ↗ — a public, browsable set of real .mzpeak files converted from open datasets across vendors, instruments, and modalities (LC-/GC-MS, imaging MS, and studies shipping SDRF / ISA sample metadata), each alongside its original. Every file opens directly in a browser viewer over HTTP range requests.

NOTE — Validator follows the API specification

The conformance validator is developed alongside the common cross-language API (see Implementations); semantic validation follows mzML precedent.

Chapter 21

Glossary

A reference for terms used throughout this specification.

Apache Arrow An in-memory columnar data model that shares Parquet’s columnar abstraction; Arrow and Parquet schemas map to one another and are straightforward to convert between.

Apache Parquet The strongly-typed, compressed, columnar on-disk format in which each facet of an mzPeak archive is stored. See Anatomy of a Parquet file.

Centroid A spectrum (or peak list) reduced to discrete m/z–intensity peaks, as opposed to profile data.

Chromatogram A measurement over time (for example a total ion current or extracted-ion trace); one of the mzPeak entity types. See Entity Type.

Chunked layout A signal-data layout that cuts the sorted main axis into non-overlapping chunks (for example fixed m/z windows) — recording each chunk’s start, end, and index — to allow random access along the axis. See Chunked Layout.

Controlled vocabulary (CV) A curated set of defined terms with stable accessions, used to annotate metadata unambiguously. mzPeak uses the PSI-MS controlled vocabulary.

CURIE A compact URI of the form `prefix:reference` (for example `MS:1000559`) used to reference a controlled-vocabulary term.

Data kind An index-file field declaring a member’s semantics and expected schema family: `data_arrays`, `peaks`, `metadata`, `proprietary`, or `other`. See Data Kind.

Delta encoding An opaque transform that stores successive differences instead of absolute values, compressing sorted axes such as m/z efficiently.

Entity type An index-file field declaring what a member describes: `spectrum`, `chromatogram`, `wavelength_spectrum`, or `other`. See Entity Type.

HUPO-PSI The Human Proteome Organization Proteomics Standards Initiative, the body that governs mzPeak and maintains the PSI-MS controlled vocabulary.

imzML The mzML-derived open standard for mass-spectrometry imaging data (XML plus an `.ibd` binary sidecar).

Index file (`mzpeak_index.json`) The JSON manifest listing every archive member with its `entity_type` and `data_kind`, so a reader resolves files by meaning rather than by name. See Index File.

Ion mobility A gas-phase separation dimension (for example drift time or $1/K_0$) that may be stored as an array parallel to m/z and intensity.

m/z Mass-to-charge ratio; the primary measurement axis of a mass spectrum.

mzML The HUPO-PSI XML standard for mass-spectrometry data; mzPeak draws on its data model and reuses concepts such as controlled vocabularies where feasible.

mzPeak An open, columnar mass-spectrometry data format: a ZIP archive (or directory) of Apache Parquet facets plus a JSON index, annotated using the PSI-MS controlled vocabulary under HUPO-PSI.

Null marking A profile-data compression technique that replaces flanking zero-intensity points with null m/z and intensity values — so Parquet stores only a validity bit — reconstructing positions from a fitted m/z-spacing model. See Null Marking.

Numpress (MS-Numpress) A family of lossy and lossless numeric compression schemes for m/z and intensity arrays, usable in mzPeak as an opaque transform.

Opaque transform A per-array encoding (for example delta encoding or Numpress) applied within a chunk and recorded so a reader can decode it. Some transforms (such as certain Numpress modes) are lossy.

Packed parallel metadata tables The metadata layout that stores several related sub-tables (spectrum, scan, precursor, selected ion) side by side, linked by primary- and foreign-key index columns. See

Packed Parallel Metadata Tables.

Page index A Parquet footer structure — which mzPeak writers MUST emit — that enables random access below the row-group level.

Point layout A signal-data layout that stores arrays as-is in parallel columns alongside a repeated entity-index column. See Point Layout.

Profile A spectrum stored as a quasi-continuous trace of m/z -intensity samples, as opposed to centroid data.

PSI-MS CV The controlled vocabulary of mass-spectrometry terms maintained by HUPO-PSI; the semantic backbone of mzPeak metadata.

Row group A horizontal partition of a Parquet file — the unit of columnar compression and coarse random access.

Sorting rank An array's ordering role within a signal layout; sorting rank 0 is the sorted “main axis” around which all parallel arrays are arranged. Arrays of differing length SHOULD instead be stored as auxiliary arrays.

Total ion current (TIC) The summed intensity recorded for a spectrum; frequently traced over time as a chromatogram.

Zero-run stripping A profile-data size reduction that removes all but the first and last zero-intensity points in empty regions of a spectrum. See Zero Run Stripping.

Chapter 22

References

1. Van Den Bossche T, Alexandrov T, Bilbao A, *et al.* mzPeak: Designing a Scalable, Interoperable, and Future-Ready Mass Spectrometry Data Format. *Journal of Proteome Research*. 2025;24(11):5329–5335. doi:[10.1021/acs.jproteome.5c00435](https://doi.org/10.1021/acs.jproteome.5c00435) — the mzPeak white paper; primary reference for the format.
2. Bradner S. Key words for use in RFCs to Indicate Requirement Levels. BCP 14, RFC 2119; 1997. doi:[10.17487/RFC2119](https://doi.org/10.17487/RFC2119) — defines the MUST / SHOULD / MAY requirement levels used throughout this specification (cited above as “(2)”).
3. Martens L, Chambers M, Sturm M, *et al.* mzML — a Community Standard for Mass Spectrometry Data. *Molecular & Cellular Proteomics*. 2011;10(1):R110.000133. doi:[10.1074/mcp.R110.000133](https://doi.org/10.1074/mcp.R110.000133)
4. Teleman J, Dowsey AW, Levander F, *et al.* Numerical Compression Schemes for Proteomics Mass Spectrometry Data. *Molecular & Cellular Proteomics*. 2014;13(6):1537–1542. doi:[10.1074/mcp.O114.037879](https://doi.org/10.1074/mcp.O114.037879) — MS-Numpress.
5. Schramm T, Hester A, Klinkert I, *et al.* imzML — A common data format for the flexible exchange and processing of mass spectrometry imaging data. *Journal of Proteomics*. 2012;75(16):5106–5110. doi:[10.1016/j.jprot.2012.07.026](https://doi.org/10.1016/j.jprot.2012.07.026)
6. Dai C, Füllgrabe A, Pfeuffer J, *et al.* A proteomics sample metadata representation for multiomics integration and big data analysis. *Nature Communications*. 2021;12:5854. doi:[10.1038/s41467-021-26111-3](https://doi.org/10.1038/s41467-021-26111-3) — SDRF-Proteomics.
7. Apache Software Foundation. Apache Parquet. <https://parquet.apache.org/>
8. Apache Software Foundation. Apache Arrow. <https://arrow.apache.org/>
9. HUPO-PSI. PSI-MS Controlled Vocabulary. <https://github.com/HUPO-PSI/psi-ms-CV>

Chapter 23

Authors & Contributors

23.1 Authors

Joshua A. Klein Boston, MA, USA · joshua.adam.klein@gmail.com *Lead developer; original author of the specification and the Rust reference implementation.*

Tim Van Den Bossche Ghent University, Ghent, Belgium; VIB-UGent Center for Medical Biotechnology, VIB, Ghent, Belgium · Tim.VanDenBossche@ugent.be

Samuel Wein Institute for Bioinformatics and Medical Informatics, University of Tübingen, Germany · samuel.wein@uni-tuebingen.de

Oliver Kohlbacher Applied Bioinformatics, Dept. of Computer Science, University of Tübingen; Institute for Bioinformatics and Medical Informatics, University of Tübingen; Institute for Translational Bioinformatics, University Hospital Tübingen · oliver.kohlbacher@uni-tuebingen.de

NOTE – Author list in progress

Additional authors from the mzPeak mailing list and the HUPO-PSI working groups will be added as the specification matures.

23.2 Contributors

The format has benefited from discussion and feedback at HUPO-PSI meetings and on the mailing list, including contributions on imaging MS, ion mobility, vendor integration, and the reference implementations.

NOTE – Contributor list in progress

A full contributor acknowledgement will be compiled from the mailing list and meeting participants.

23.3 How to contribute

mzPeak is developed in the open. The most valuable contribution right now is **careful review of this specification** by readers with fresh eyes, reporting anything unclear or missing.

- File issues or suggestions: github.com/HUPO-PSI/mzPeak-specification/issues
- Reference implementation (Rust): github.com/HUPO-PSI/mzPeak

Chapter 24

Intellectual Property

24.1 Intellectual property statement

The PSI takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this document, or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Copies of claims of rights made available for publication, any assurances of licenses to be made available, or the result of an attempt to obtain a general license or permission for the use of such proprietary rights by implementers or users of this specification, can be obtained from the PSI Chair.

The PSI invites any interested party to bring to its attention any copyrights, patents, patent applications, or other proprietary rights that may cover technology required to practice this recommendation. Please address such information to the PSI Chair (see contact information on the [PSI website](#)).

24.2 Trademark

The **mzPeak** name is held in trust by OpenMS Inc. on behalf of the HUPO-PSI committee, pending establishment of a suitable legal entity. The trademark ensures that no incompatible implementation uses the mzPeak name; the standard itself remains open. Details of the trademark are described at DOI [10.5281/zenodo.20054899](https://doi.org/10.5281/zenodo.20054899).

24.3 Copyright notice

Copyright © 2026 by the Human Proteome Organization (HUPO) Proteomics Standards Initiative (PSI), under the [CC-BY-ND 4.0 license](#).